

Sonstige Themen

Operatoren überladen

- Unäre Operatoren (+, -, !, ~, ++, --, true, false)

```
public static Zug operator ++(Zug wagon)
```

- Binäre Operatoren (+, -, *, /, %, &, |, ^, <<, >>)

```
public static Zug operator +(Zug wagon, Wagon wagon)
```

```
public static Zug operator +(Wagon wagon, Zug zug)
```

```
public static Zug operator +(Wagon wagon1, Wagon wagon2)
```

- Vergleichsoperatoren (==, !=, <, >, <=, >=)

```
public static bool operator ==(Wagon wagon1, Wagon wagon2)
```

Enumerator für eine Klasse definieren

```
public class Zug : IEnumerable           //Interface implementieren
public List<Wagon> Wagons { get; set; }

IEnumerator IEnumerable.GetEnumerator()
{
    foreach(var item in Wagons)
    {
        yield return item;
    }
}
```

```
Zug ICE = new Zug("ICE");
foreach(Wagon item in ICE)
```

Indexerproperties

```
public class Zug
public List<Wagon> Wagons { get; set; }

public Wagon this[int index] //Indexer-Property
{
    get { return this.Wagons[index]; }
    set { this.Wagons[index] = value; }
}
```

```
Zug ICE = new Zug("ICE");
Wagon SpeiBewagen = ICE[3]; //Zugriff erfolgt wie bei Array
```

Erweiterungsmethoden

```
public static int GetQuersumme(this int zahl)
{
    int summe = 0;
    string zahlAlsString = zahl.ToString();
    for(int i=0;i<zahlAlsString.Length; i++)
    {
        summe += (int)char.GetNumericValue(zahlAlsString[i]);
    }
    return summe;
}
```

This zeigt an für welche Klasse eine Erweiterungsmethode definiert werden soll

```
Console.WriteLine(10232.GetQuersumme()); //Gibt 8 zurück
```

Linq (Language Integrated Query)

- Sammlung von Erweiterungsmethoden zur Abfrage und Sortierung von Elementen innerhalb von Aufzählungstypen (Lists, Arrays, Stacks, Queues ...)
- Enthält auch neue Syntax, welche an SQL-Syntax angelehnt angelehnt ist

```
using System.Linq;  
string [] Städte = new string[] { "Leipzig", "Hamburg", "Hannover" };  
var mitH = from stadt in Städte  
           where stadt.StartsWith("H")  
           select stadt;
```

Ergebnis: String-Liste mit Hamburg und Hannover

```
//Linq-Syntax entspricht dieser Schreibweise  
var mitH = Städte.Where(stadt => stadt.StartsWith("H"));
```

Timer

- führt in einem angegebenen Zeitintervall ein Event aus
- benötigt den Namespace System.Timers;

```
int countdownNumber = 100;
```

```
Timer timer = new Timer();  
timer.Elapsed += new ElapsedEventHandler(Countdown);  
timer.Interval = 1000;  
timer.Start();
```

```
void Countdown(Object sender, ElapsedEventArgs args)  
{  
    countdownNumber--;  
    Console.WriteLine(countdownNumber);  
    if (countdownNumber <= 0)  
    {  
        timer.Stop();  
    }  
}
```